



Lecture #08: 디자인 패턴 소개

객체지향설계와패턴01강좌
2026.03.31(화)

학습 목표

- 디자인 패턴이란?
- 디자인 패턴의 필요성
 - 디자인 패턴의 비적용과 적용
- 디자인 패턴의 유형
 - 패턴에 적용되는 공통된 형식

8.1 패턴이란?

- 웹스터 사전의 정의
 - “모방을 위하여 제안된 형식이나 모델” - 웹스터 9판
- 소프트웨어 분야에서의 패턴이란?
 - 비슷한 상황을 겪은 경험을 기초로 공통된 문제와 솔루션을 기술해 놓은 것
- 책들의 분류
 - 문학 작품에 드러나는 전형성이 있는 것처럼 소프트웨어를 만들 때 그러한 개념을 부여해보자.
 - 역사서, 로맨틱 코미디 소설, 탐정물, 철학서
- 부품의 이름
 - 왜 그거 있잖아.. 콘덴서랑 캐파 몇 개랑 어떻게 붙이고 ... ➔ 아!! 파워 서플라이..

패턴의 역사

- 1970년대 Christopher Alexander의 패턴 언어로 출발
- 벽돌집 건축에서
 - North South Axis
 - West Facing Entrance Down The Slope
 - Two Floors
 - Hay Loft At The Back
 - Bedrooms In Front
 - Garden To The South
 - Pitched Roof
 - Half-Hipped End
 - Balcony Toward The Garden
 - Carved Ornaments

패턴의 역사

- 1987년에 Kent Beck와 Ward Cunningham이 OOPSLA 학회에서 *Using Pattern Languages for Object-Oriented Programs* 라는 제목의 논문 발표
- UI 설계에 Alexander의 방법 적용
 - Window Per Task
 - Few Panes Per Window
 - Standard Panes
 - Short Menus
 - Nouns and Verbs
- 1995년에 Erich Gamma, Richard Helm, Ralph Johnson, and John Vlissides이 *Design Patterns: Elements of Reusable Object-Oriented Software* 책 출간
 - 23개의 패턴
 - Gang of Four (GoF)로 불림

필요성

- 여러 명이 한 팀을 이루어 S/W 개발할 때, 무엇을 매개로 토의할까?

~~Source Code~~

UML diagrams

특정 코드 전체를
설명할 수 있는 단어

팀원 모두 Singleton 패턴의 구조와 동작방식을 알고 있다면,
A: “이건 Singleton 패턴으로 작성되어 있어”
B: “아 그래 그렇다면 이렇게 사용하면 되겠군.”

최근의 소프트웨어 개발 추세

- 구현 위주에서 설계 위주로
 - 소프트웨어 크기가 커짐
 - 여러 명, 여러 팀의 협동 개발필요
 - 객체지향적 방법론에서 설계가 잘된다면 구현은 어느 정도 단순작업화 됨
- 구현 단계에서는 “Code Reuse” (Code Library, Component)를 통하여 생산성과 효율성을 높일 수 있다.
- 디자인단계에서의 재사용은
→ “Design Patterns”

디자인 패턴이란?

- 일반적인 설계문제를 해결하기 위하여 짜여진 상호 협력하는 객체들과 클래스 들에 대한 기술[GoF]
- 자주 발생하는 설계상의 문제를 해결하기 위한 반복적인 해법 [Smalltalk Companion]
- 반복되는 구조를 설계할 때 설계를 재활용하는데 초점을 두는데 비하여 프레임워크는 세부 설계와 구현에 초점을 두고 있다[Coplien & Schmidt]

패턴을 사용하면?

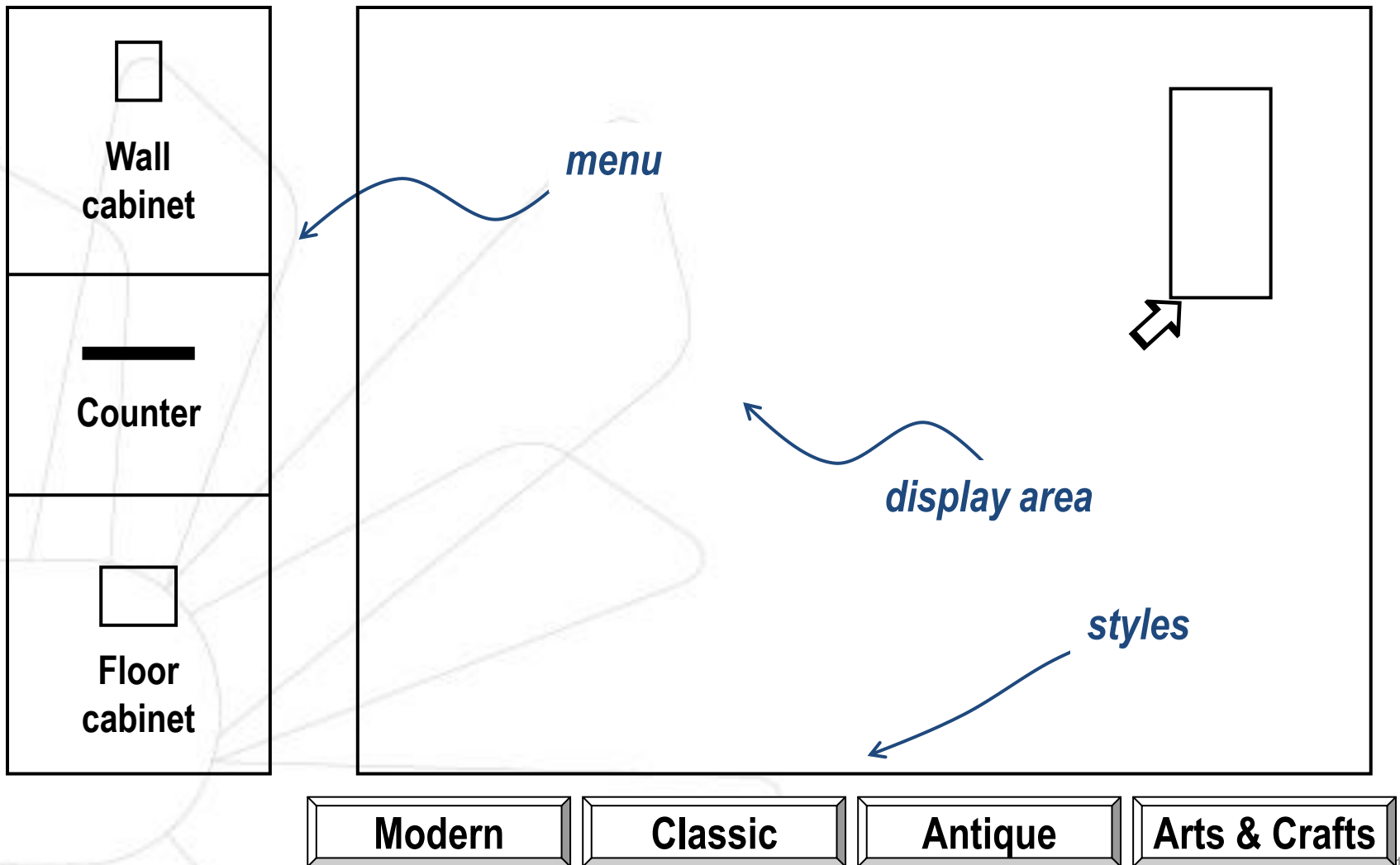
- 전문가의 경험이므로 적용하여 좋은 설계가 되도록 도와줌
 - 코드가 더 견고하게 함
 - 재사용을 용이하게 함
- 여러 번의 시행착오를 거치면서 비슷한 역할의 클래스를 자주 쓰게 되어 유형화 됨
 - 설계에 대한 의사소통이 잘 됨
 - 설계 Know-how가 늘어감

설계 목표

- 재사용성, 융통성, 유지보수성
 - 융통성 있는 설계의 재사용
 - 코드를 일반화
 - 다른 클래스에 의존하는 것을 최소화
- 강인성
 - 신뢰성 있는 설계의 재사용
 - 강인한 부품을 재사용
- 충분함, 정확성
 - 모듈화 된 설계
 - 신뢰성 있는 부품의 재사용

8.2 패턴의 필요성

KitchenViewer Interface



사용 사례: 레이아웃 설계

사용사례 : 레이아웃 설계

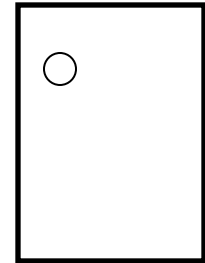
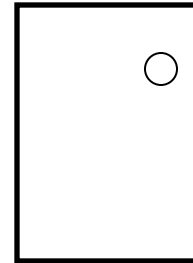
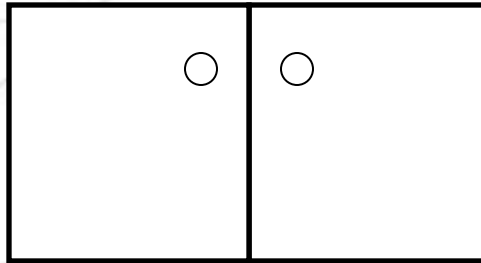
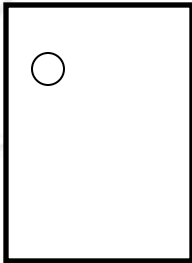
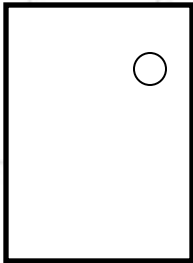
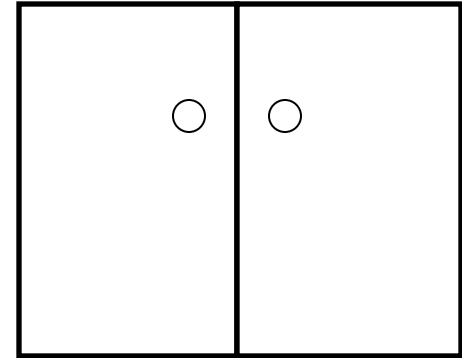
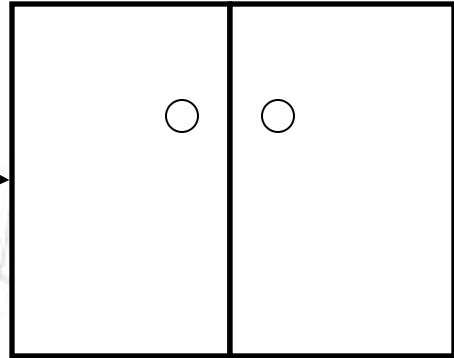
액터 : 사용자

이벤트 :

1. 사용자가 'wall cabinet' 아이콘을 클릭
2. 응용 프로그램이 워크 영역에 붙박이장을 보여줌
3. 사용자가 붙박이장의 크기를 조정
4. 붙박이장을 끌어 상단에 위치시키고 마우스를 릴리스
5. 계속 붙박이장을 만들어 크기 조절하고 원하는 곳에 이동
6. 'floor cabinet(바닥장)'을 선택하여 바닥장을 만들고 원하는 위치에 이동
7. ...

KitchenViewer

Countertop



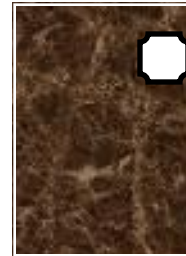
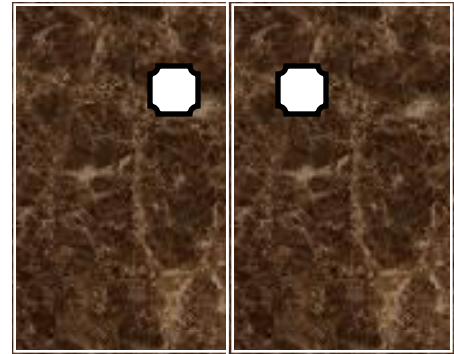
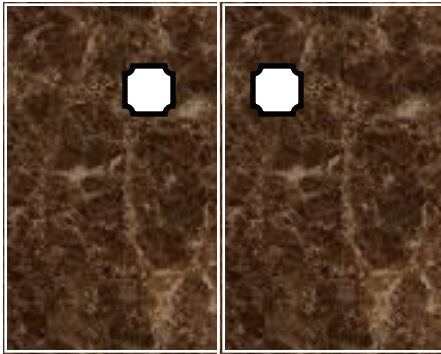
Modern

Classic

Antique

Arts & Crafts

Antique Style의 선택



Modern

Classic

Antique

Arts & Crafts

피해야 할 코딩 스타일

- 렌더링

```
Counter counter = new Counter();  
draw (counter);
```

- 상판(Counter)의 타입이 런타임에도 바뀔 수 있음
- 따라서 다음과 같은 스타일로 바꾸어야 함

```
renderKitchen(myStyle);
```

*어플리케이션이 런타임에 객체의 패밀리로 구성된다면 여러 가지
주어진 스타일에서 선택할 수 있도록 하여야 함*

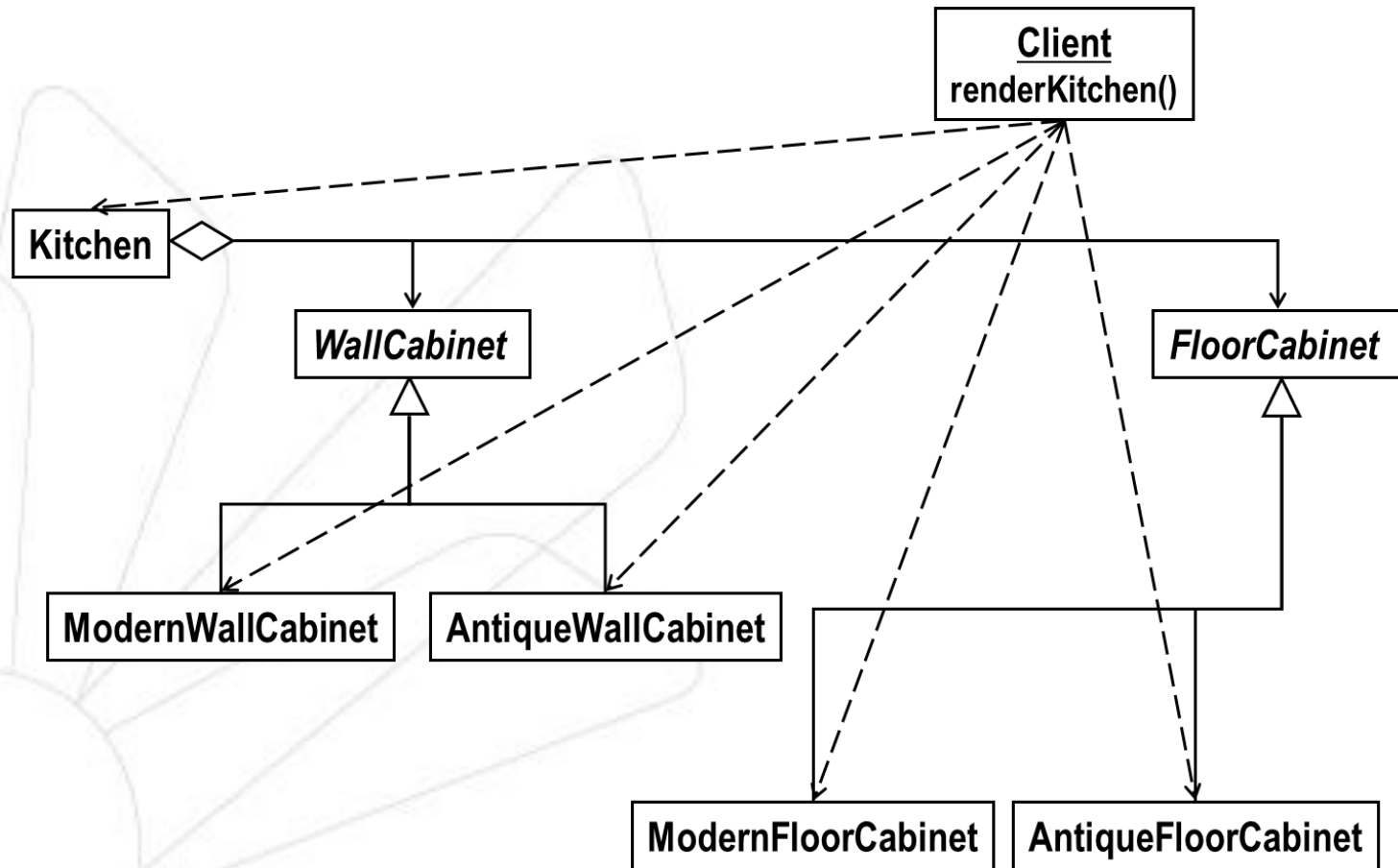
디자인 패턴을 적용하지 않은 코딩

```
// Create the antique wall cabinets
AntiqueWallCabinet antiqueWallCabinet1 = new AntiqueWallCabinet();
AntiqueWallCabinet antiqueWallCabinet2 = new AntiqueWallCabinet();

// Create the antique floor cabinets
AntiqueFloorCabinet antiqueFloorCabinet1 = new AntiqueFloorCabinet();
AntiqueFloorCabinet antiqueFloorCabinet2 = new AntiqueFloorCabinet();

// Create the kitchen object, assuming the existence of add() methods
Kitchen antiqueKitchen = new Kitchen();
antiqueKitchen.add(antiqueWallCabinet1, ... );
antiqueKitchen.add(antiqueWallCabinet2, ... );
...
antiqueKitchen.add(antiqueFloorCabinet1, ... );
antiqueKitchen.add(antiqueFloorCabinet2, ... );
...
// Render antiqueKitchen
...
```

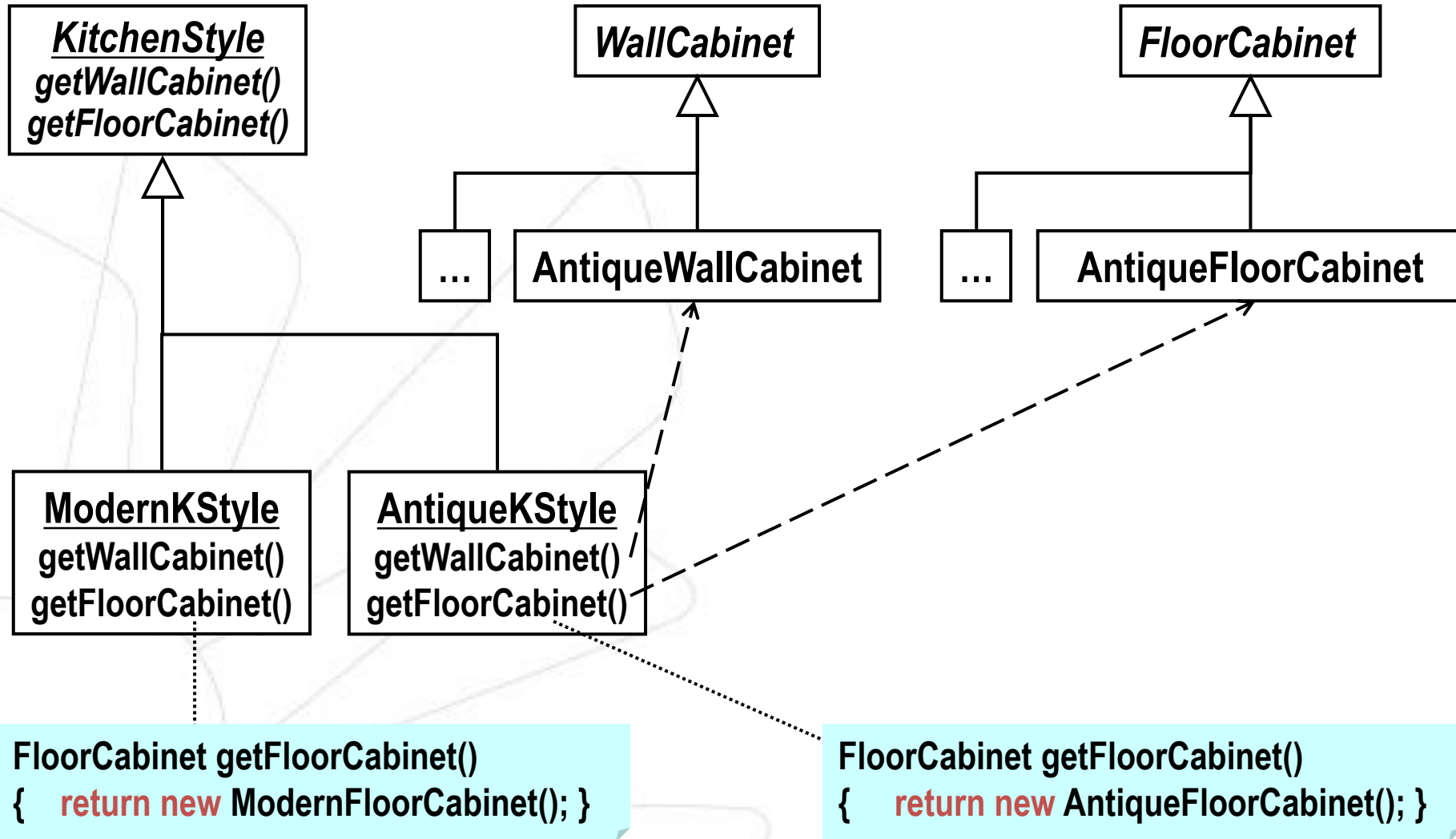
디자인 패턴을 적용하지 않은 UML



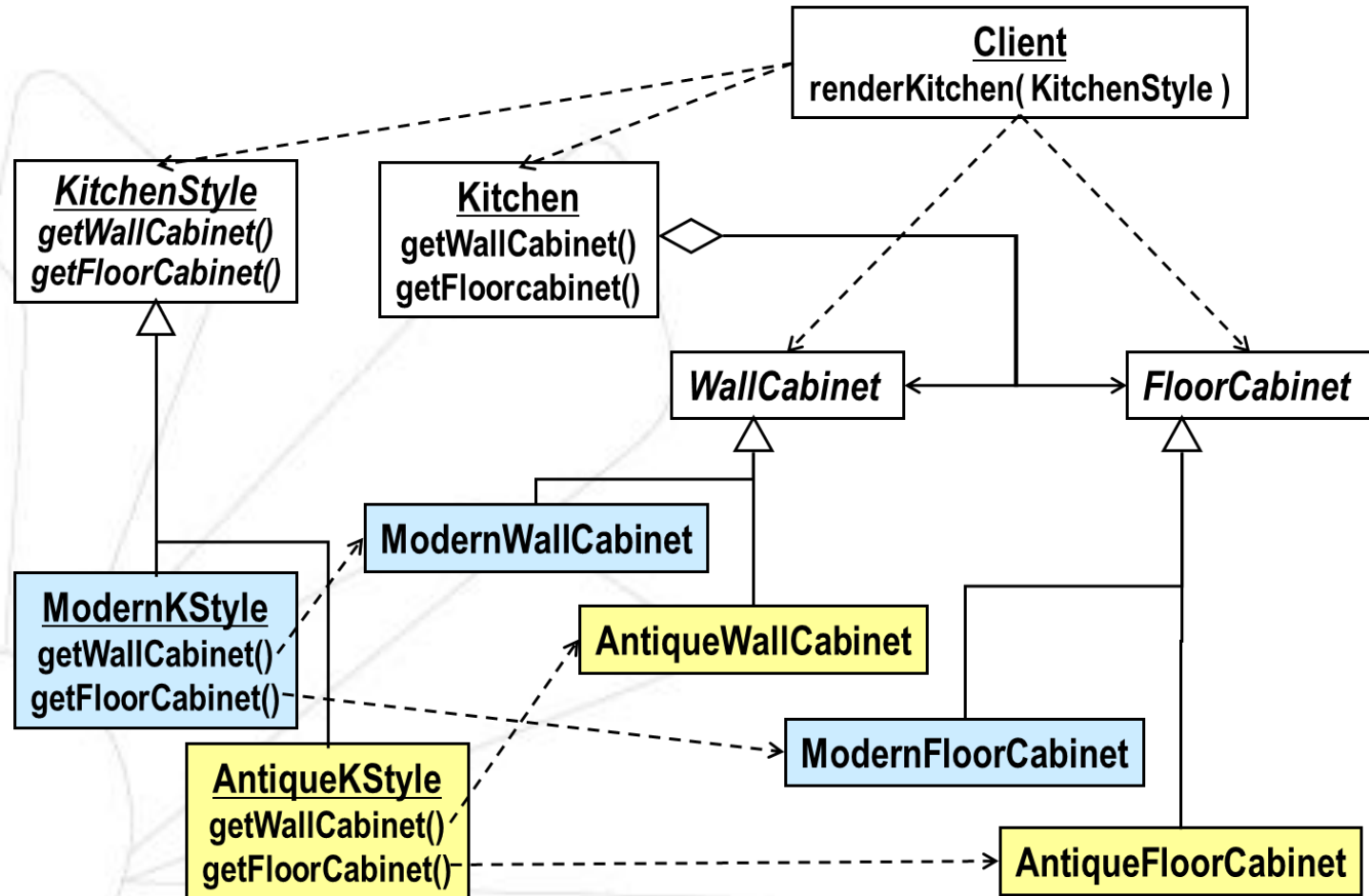
디자인 패턴을 적용한 사례

- 디자인 목표
 - 여러 가지 키친 스타일을 런타임에 생성할 수 있도록 융통성 있는 설계가 되어야 함
- 키친 스타일을 직접 생성하는 방법
`new AntiqueWallcabinet();` // 엔틱에만 적용
- 스타일 파라미터로 위임하는 코드
`myStyle.getWallCabinet();` // 런타임에 스타일이 선택

추상 팩토리 아이디어



추상 팩토리 설계 패턴 적용



객체지향 설계에서의 문제

- 하드웨어 장치의 조건을 모니터링 하기 위하여 온도센서를 사용하는 시스템을 가정
- 첫 번째 모델은 TempTek, Inc. TS7000 센서 사용
- 센서 인터페이스를 위한 다음 클래스 제공

```
class TS7000 {  
    native double getTemp();  
    ...  
}
```
- 모니터링 코드

```
TS7000 sensors[ ] = new TS7000[...];  
double sum = 0.0;  
for (int i = 0; i < sensors.length; i++)  
    sum += sensors[i].getTemp();  
double meanTemp = sum / sensors.length;
```

객체지향 설계에서의 문제

- 두 번째 모델은 TS7000과 다른 회사(Thermon)의 센서 제품도 병행 사용
- 다른 센서 SuperTemp는 다음과 같은 인터페이스 제공

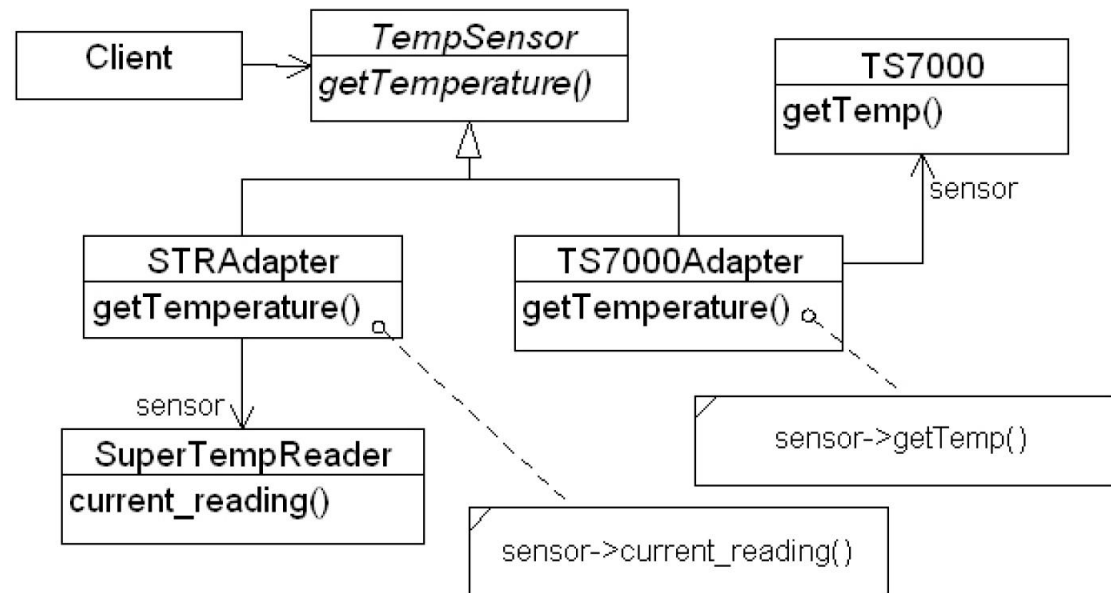
```
class SuperTempReader {  
    //  
    // NOTE: temperature is Celsius tenths of a degree  
    //  
    native double current_reading();  
    ...  
}
```

- 센서를 읽기 위한 불편한 방법

```
for (int i = 0; i < sensors.length; i++) {  
    if (sensors[i] instanceof TS7000)  
        sum += ((TS7000)sensors[i]).getTemp();  
    else  
        // Must be a SuperTemp!  
        sum += ((SuperTempReader)sensors[i]).current_reading() * 10;  
}
```

해결책

- 문제
 - 미래 다양한 센서의 추가
 - 온도 리포팅 메소드의 이름이 다양
- 해결책 – Adapter 패턴



해결책

```
abstract class TempSensor {  
    abstract double getTemperature();  
}  
class STRAdapter extends TempSensor {  
    public double getTemperature() {  
        return sensor.current_reading() * 10;  
    }  
    ... constructor, etc. ...  
}  
class TS7000Adapter extends TempSensor {  
    public double getTemperature() {  
        return sensor.getTemp();  
    }  
    ... constructor, etc. ...  
}
```

- 평균 온도 계산을 위한 다른 버전:
double sum = 0.0;
for (int i = 0; i < sensors.length; i++)
 sum += sensors[i].getTemperature();

Adapter 패턴

- 의도
 - 클래스의 인터페이스를 클라이언트가 예상하는 형태로 변환하고자 할 때 ADAPTER는 호환되지 않는 인터페이스를 바꾸어 같이 협력하도록 만들어 줌.
- 적용
 - 평균 온도 구하는 클라이언트 프로그램은 센서 종류에 독립적인 TempSensor 인터페이스로 구현할 수 있음(유통성)
 - 추가 센서가 있는 경우 TempSensor 서브 클래스만 간단히 추가
- 재사용을 위하여 Meyer의 Open-Closed 원리 적용

패턴의 이점

- 디자인 패턴의 실질적 이점은?
- 문제 해결 방법에 관한 간단한 언급
 - “getTemperature 메소드를 가진 슈퍼클래스를 만들고 서브 클래스를 상속하여 각 제품의 클래스 인스턴스를 갖게 하는 것이 어떨까.....”

라는 대화 대신에

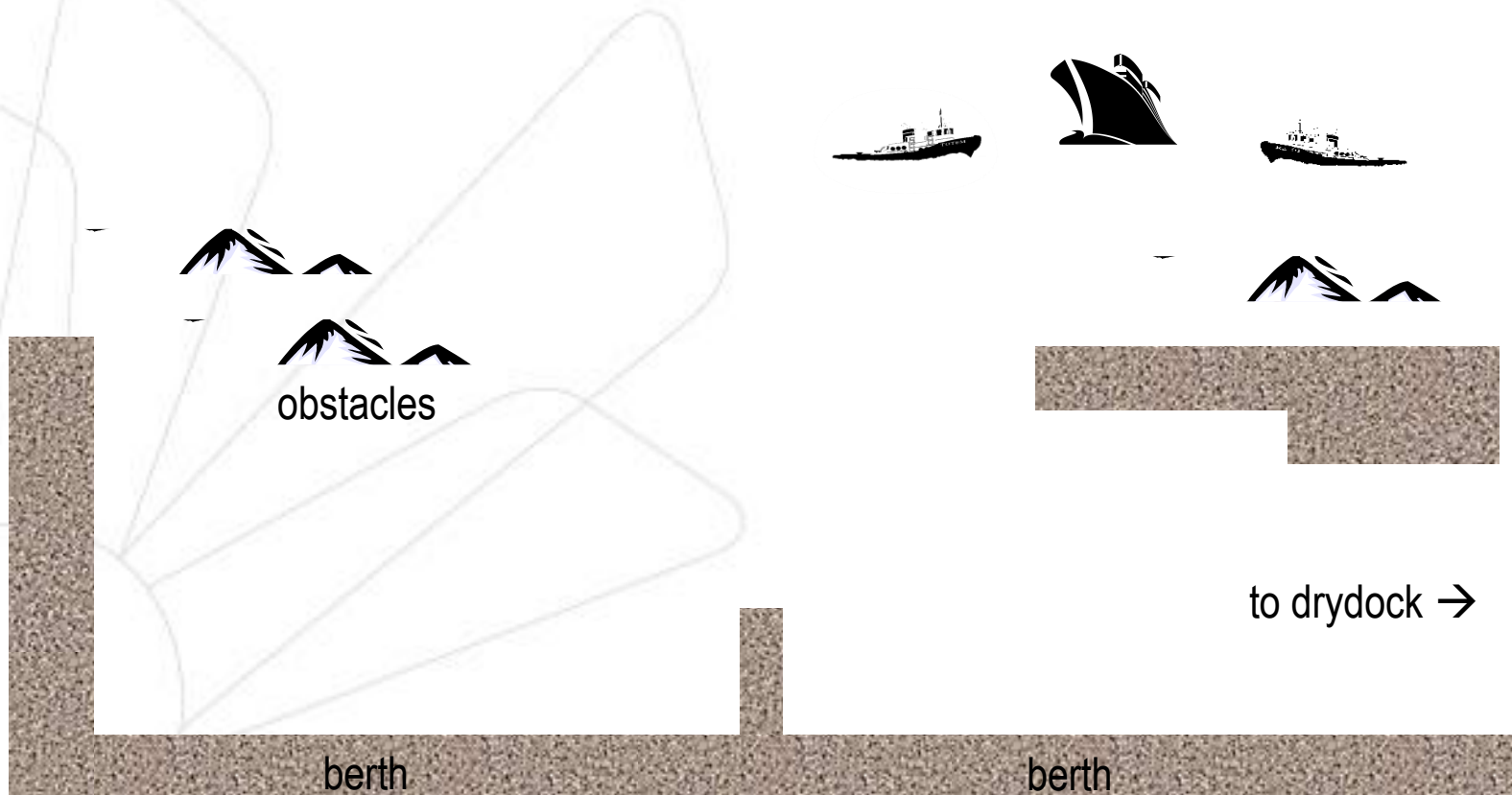
- “여기에는 **Adapter 패턴**을 써보자.”
- 패턴은 아주 새로운 것이 아니라 비슷한 문제를 다루면서 얻은 노하우가 잘 정리된 것

디자인 패턴 맛보기

- 컴포지트
 - 개별 객체와 객체의 묶음을 같은 방식으로 취급하고 싶을 때
- 옵서버
 - 1-대-다의 객체 관계에서 객체 하나의 상태변화를 다른 연관 객체들에게 자동으로 통지하고 변경하고 싶을 때
- Flyweight
 - 객체의 부담을 줄이기 위하여 작은 객체들로 나누고 공유하기 위하여
- Iterator
 - 객체집합에 있는 요소들을 내부 표현에 구애 받지 않고 순차적으로 접근하고 싶을 때
- Façade
 - 서브시스템의 인터페이스로 통합된 인터페이스를 제공하고 싶을 때

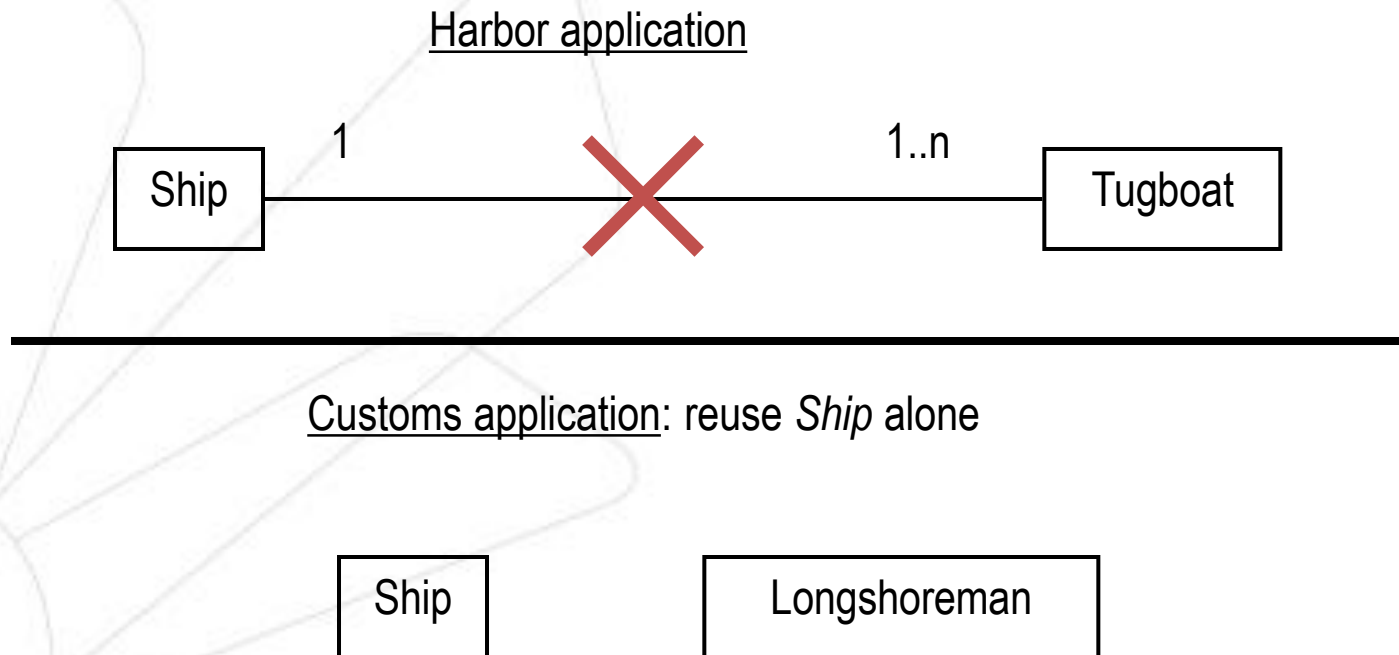
행위 패턴 - Mediator

- 객체 사이의 관계에 대한 패턴



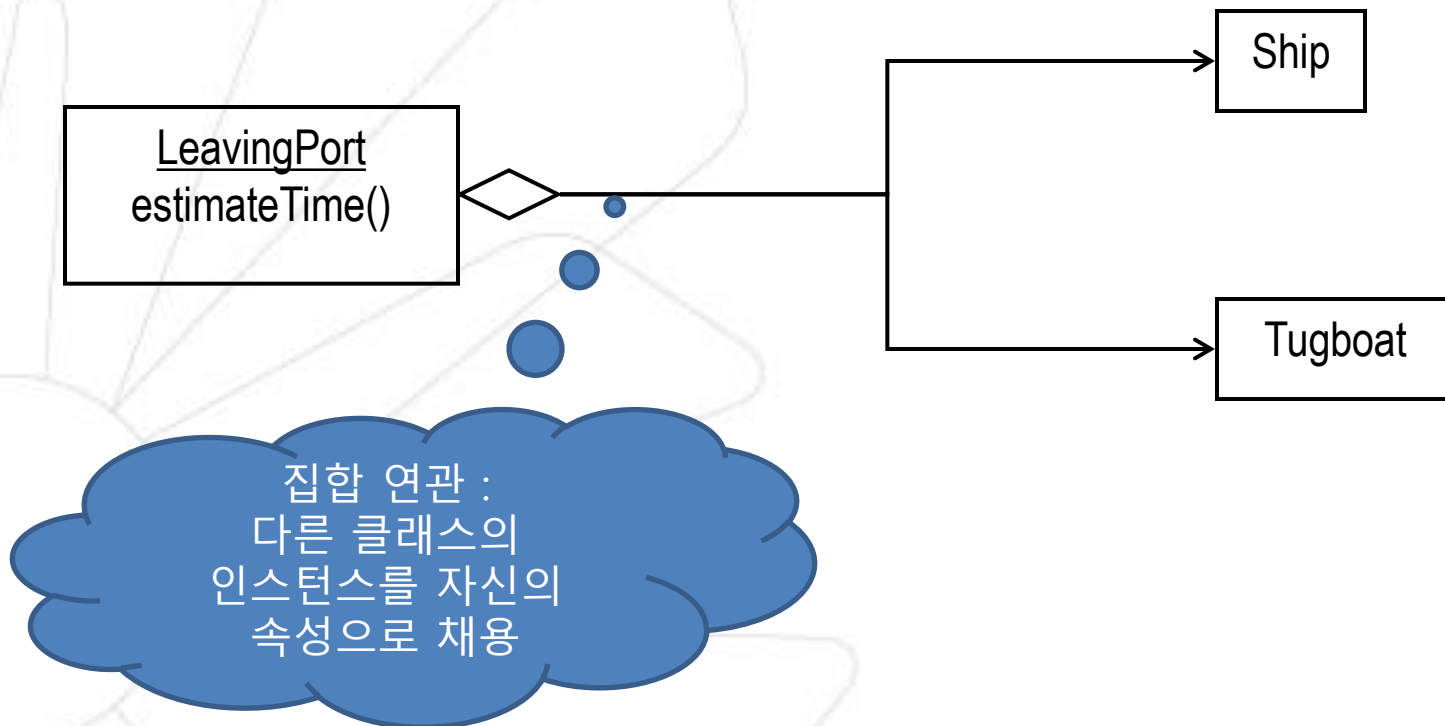
디자인 목표

- Ship과 Tugboat의 의존성을 피하도록
 - 두 클래스를 같이 사용하는 어플리케이션이 많을 것임

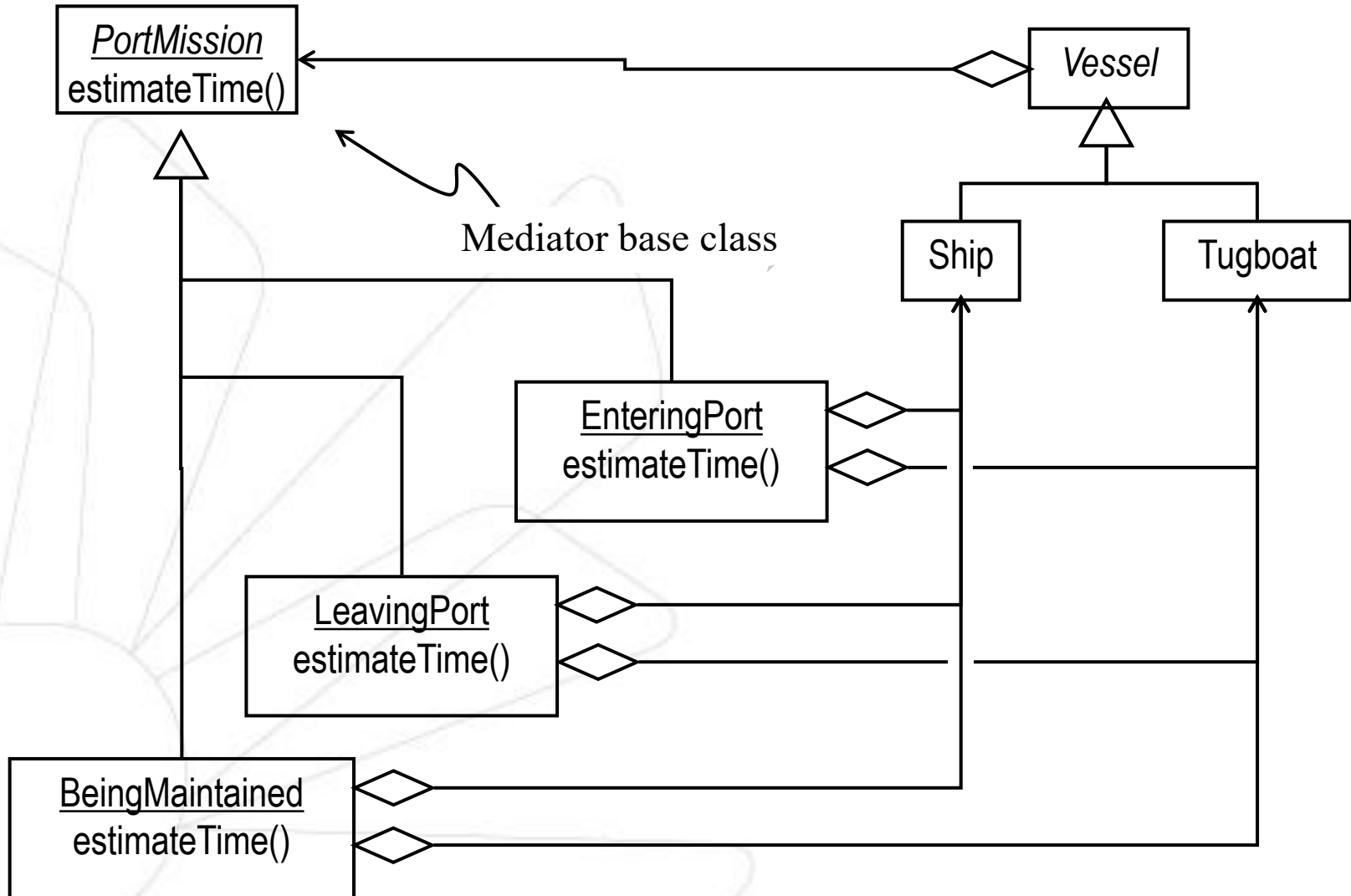


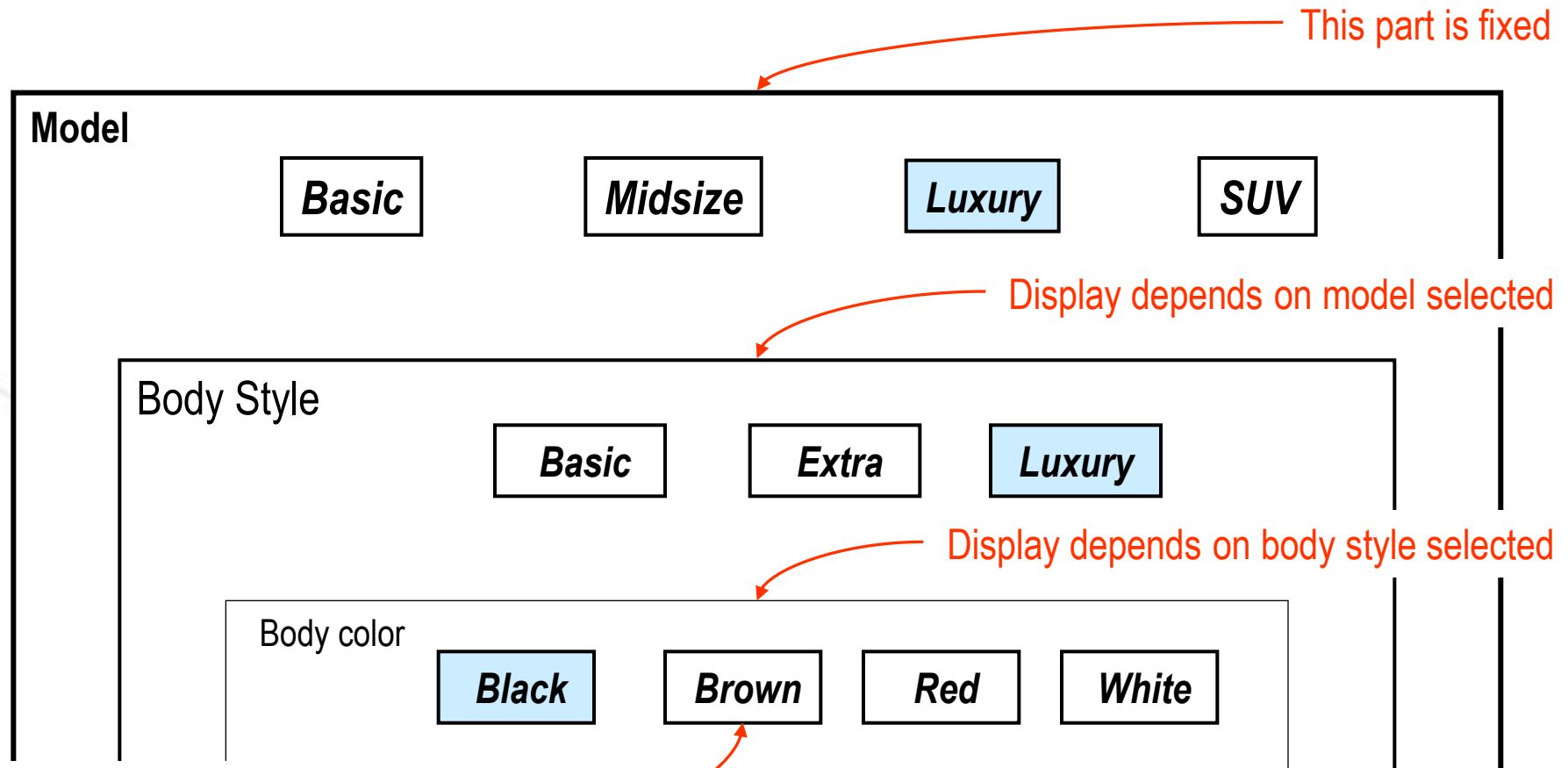
중재자 패턴의 도입

- 연관된 행위와 객체 자체를 분리하기 위하여
 - 중간에 중재하는 클래스(mediator)를 도입



중재자 패턴이 적용된 항구 문제





Assume that each of these will eventually be a small, expensive animation.

A Design Problem Solvable by *Chain of Responsibility*



You have requested to view a luxury model with luxury body style, and black body color.

패턴의 특징

- 관점(view) – 패턴을 나타내는 방식
 1. 정적: 클래스 모델(빌딩 블록)
 2. 동적: 순차 다이어그램, 상태 다이어그램(오퍼레이션)
- 수준(level) – 패턴의 분석
 1. 패턴의 핵심을 기술하는 추상적 수준
 2. 자세한 패턴 실체를 나타내는 수준
- 역할(role) – 패턴 사용에서의 역할
 1. 디자인 패턴 어플리케이션 자체
 2. 디자인 패턴을 사용하는 클라이언트
 3. 초기화 또는 제어하는 셋업 코드

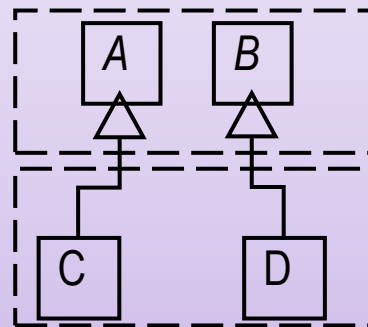
패턴 사용에서의 역할

1. Client role

2. Setup role

3. Role: Application of the design pattern

A. Static viewpoint

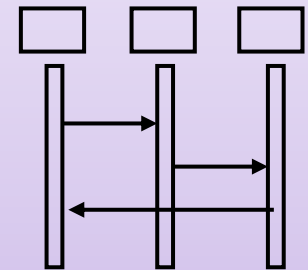


(class model)

(i) Abstract level

(ii) Concrete level

B. Dynamic viewpoint



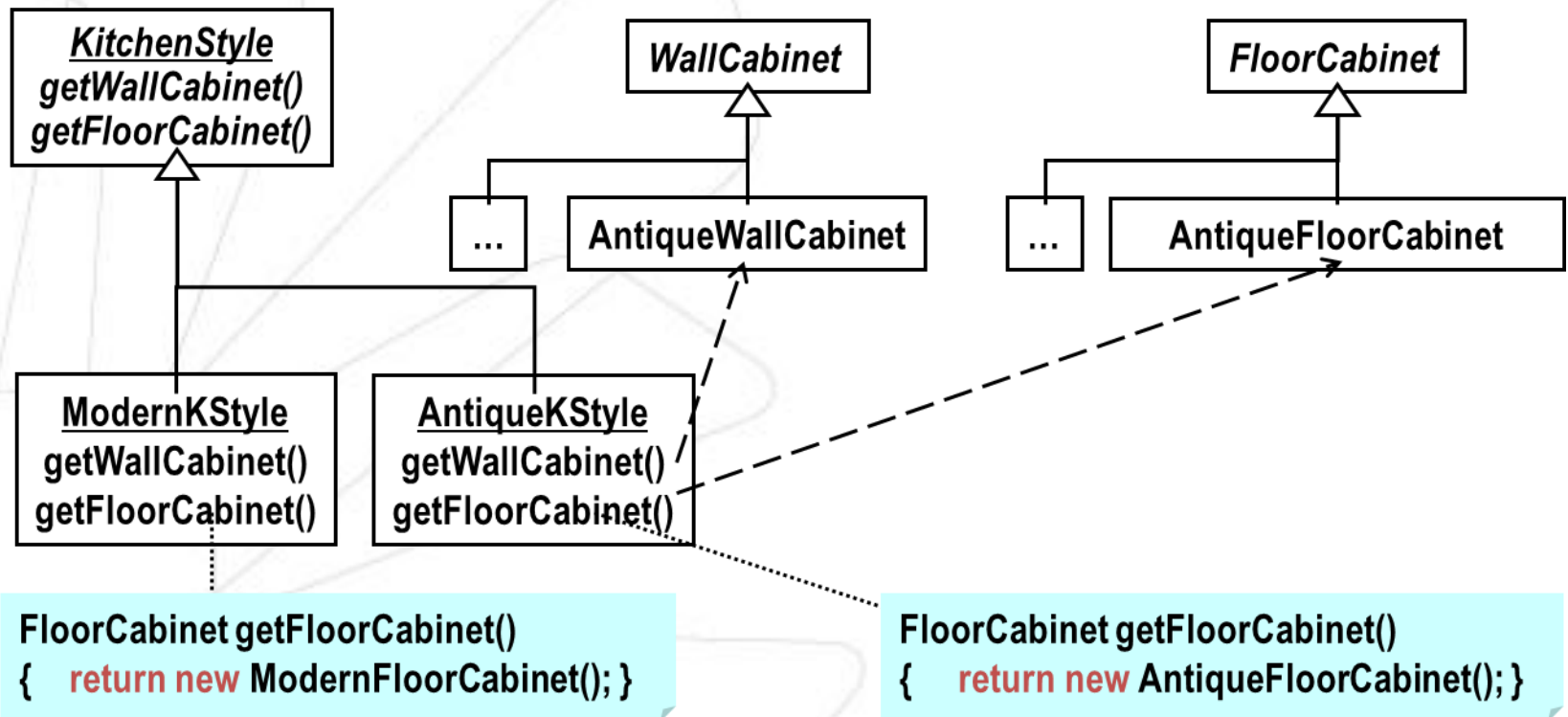
(sequence or state diagram)

(class or classes)

-----> : Reference direction

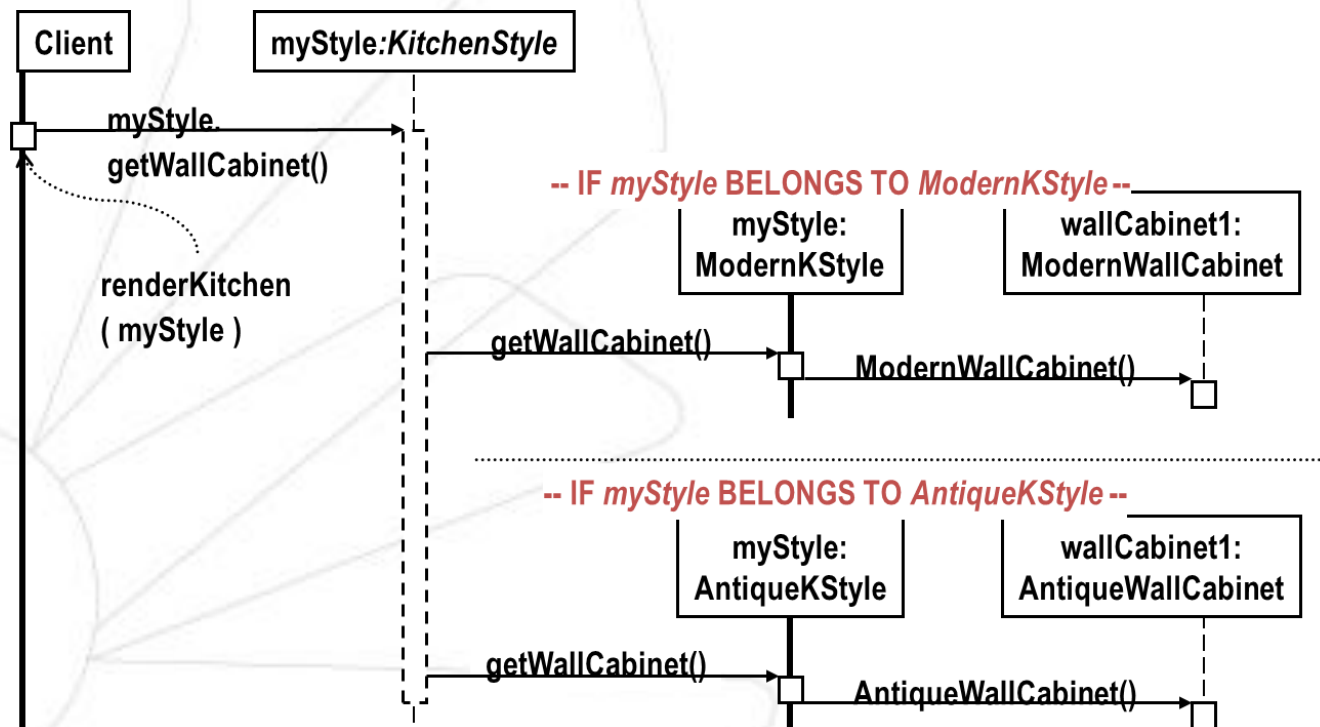
패턴을 표현하는 두 가지 관점

- 정적인 측면
 - 패턴을 이루는 클래스들과 이들의 정적인 관계를 표현
 - 클래스 다이어그램

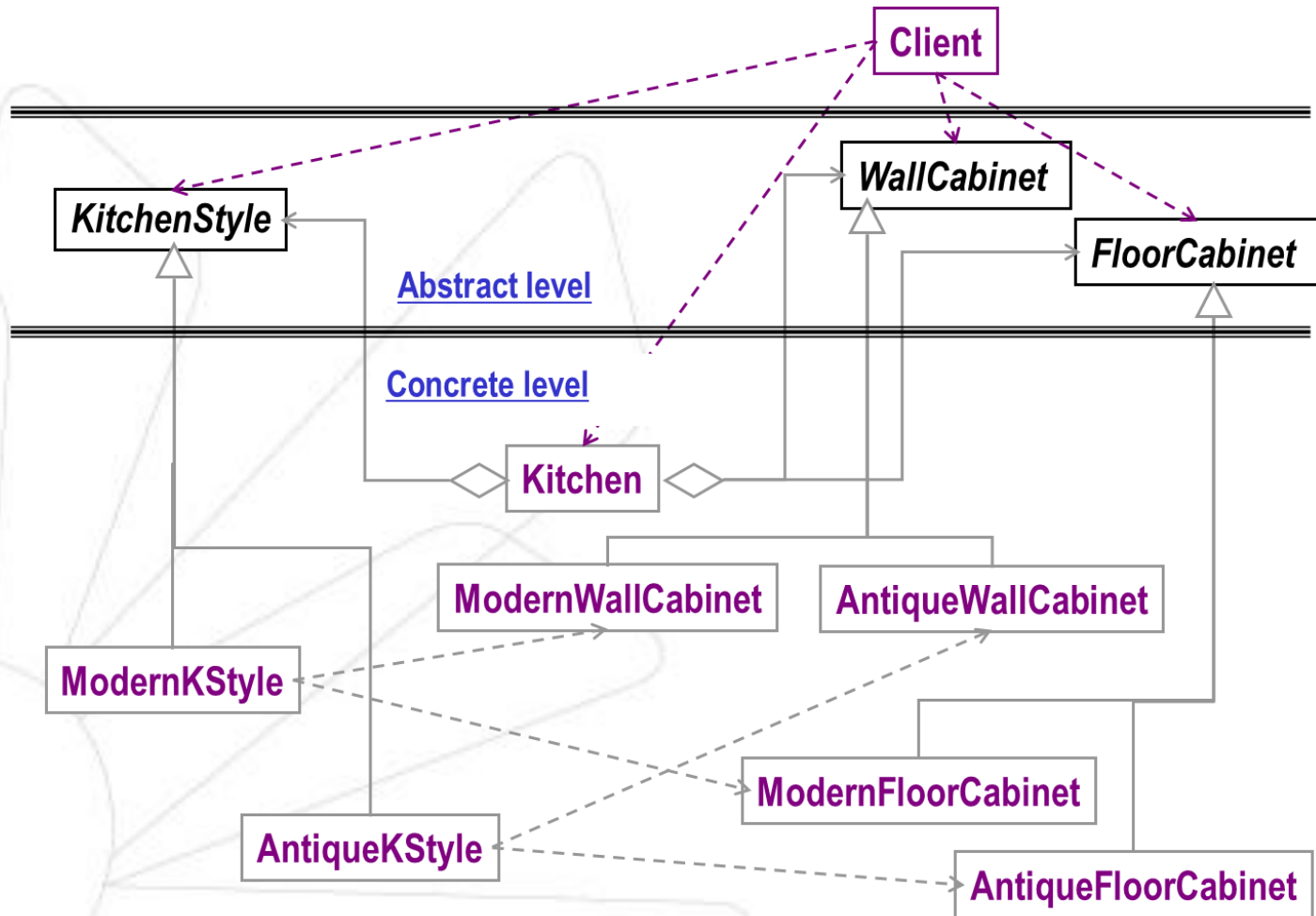


패턴을 표현하는 두 가지 관점

- 동적인 관점
 - 기능이 어떤 순서로 호출되고 실행되는지를 나타냄
 - 순차 다이어그램 등



패턴의 두 가지 수준



패턴에 관련된 세 가지 역할

2. Client Role

Interacts with the design pattern only through its interface

DPClient

1. Design Pattern Application

*Interface to the pattern
(frequently abstract;
possibly several classes)*

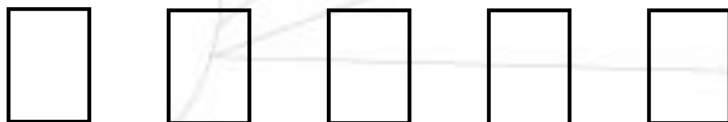
DPInterface

3. Setup Role

No limitations



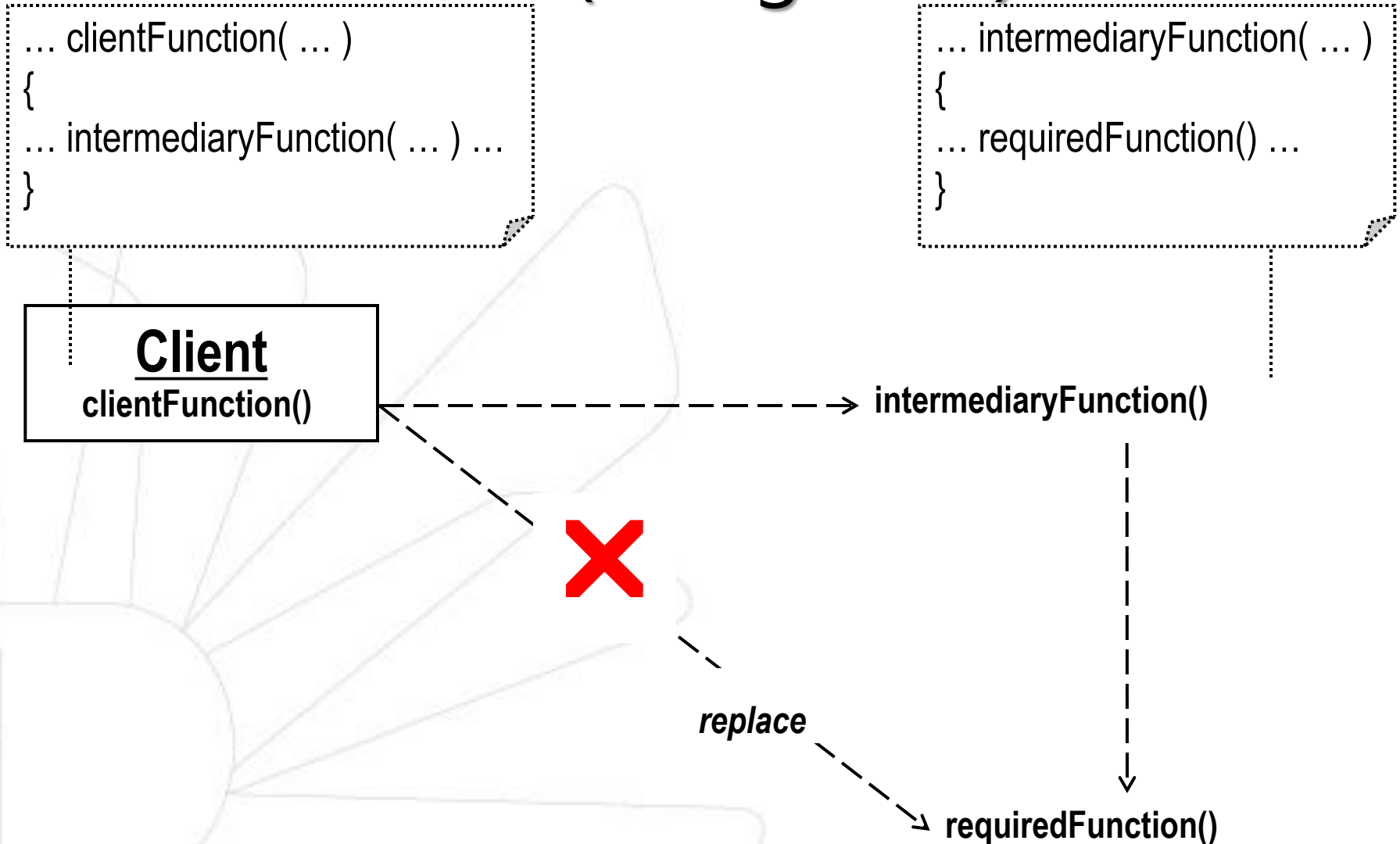
Rest of the Application



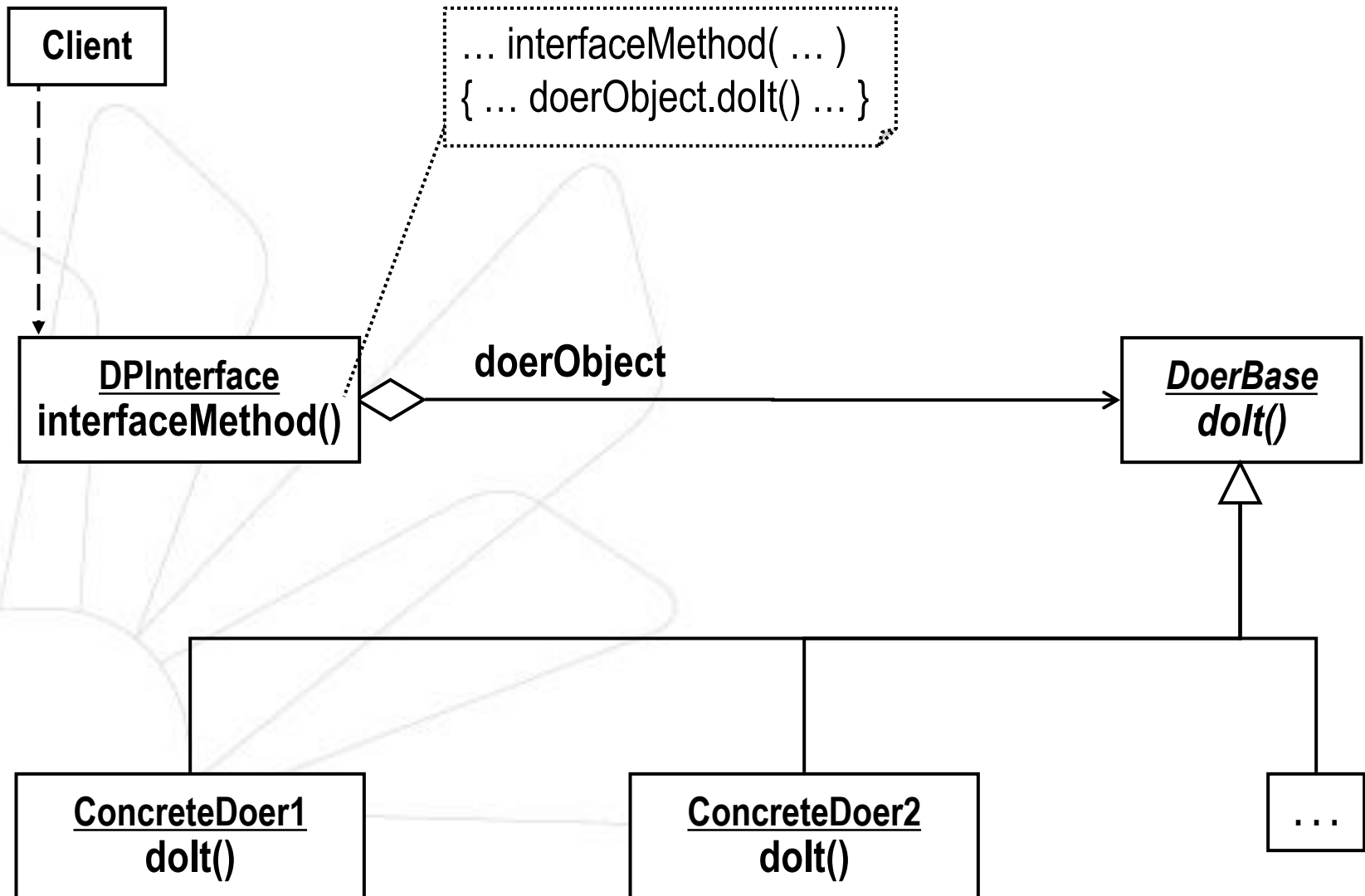
세 가지 역할

- 어플리케이션
 - 특정 패턴이 적용됨
 - KitchenViewer에서 추상 클래스의 인터페이스들
 - KitchenStyle, Kitchen, WallCabinet, FloorCabinet ...
- 클라이언트
 - 패턴을 사용하는 입장
 - KitchenViewer에서 renderMethod()
- 셋업
 - 어플리케이션 코드를 런타임에 초기화, 변경
 - KitchenViewer에서 스타일 버튼을 누르면 KitchenStyle 객체가 생성되고 이를 파라미터로 renderKitchen() 함수를 호출

위임(delegation)



디자인 패턴 형식 1 - 위임

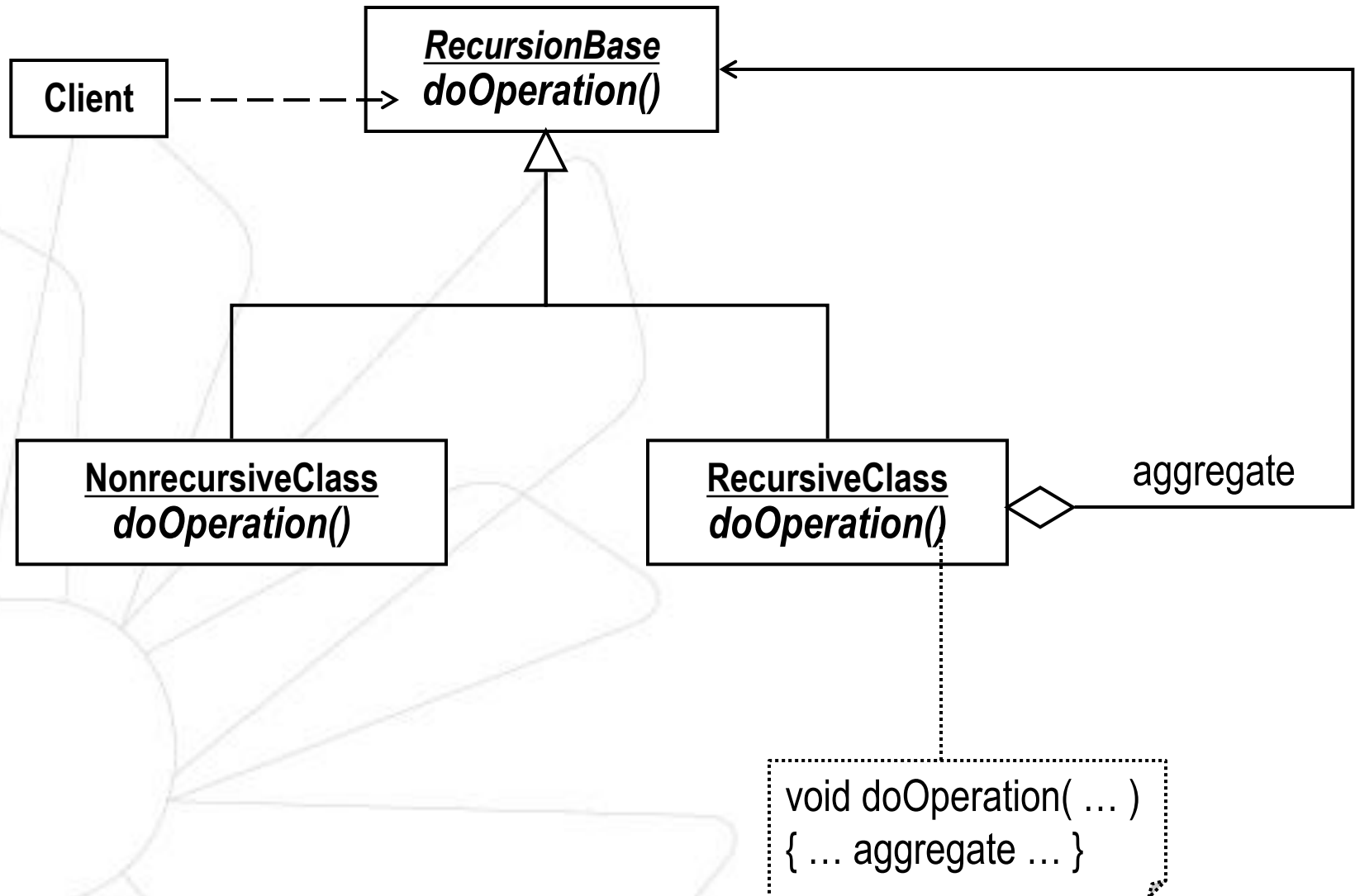


DPInterface

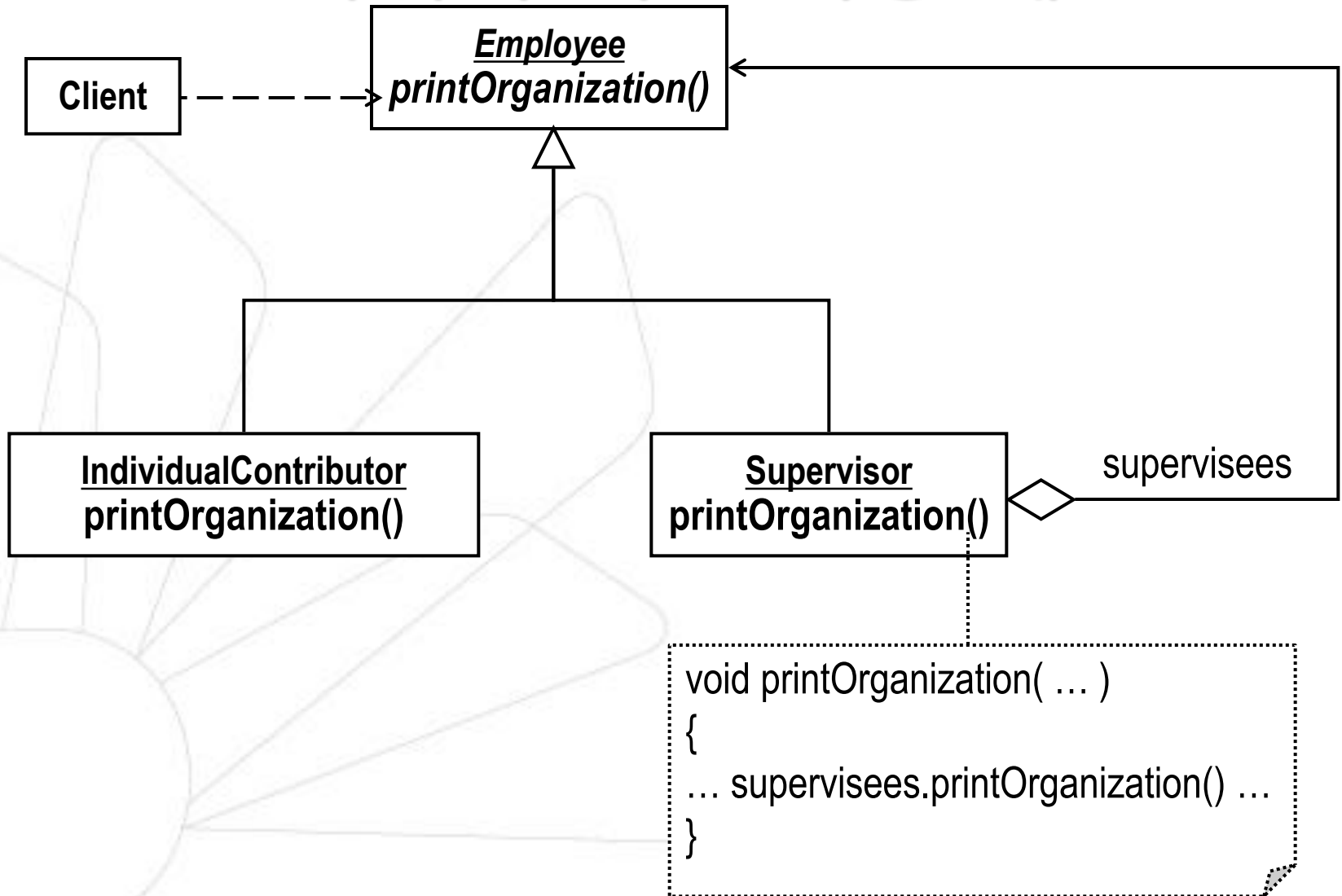
Class DPInterface

```
{
    DoerBase doerObject;
    . . .
    public void interfaceMethod()
    {
        doerObject.doIt();
    }
}
```

디자인 패턴 형식 2 – 재귀적 구성



재귀적 패턴 적용 예



디자인 패턴의 표현 형식

- 디자인 패턴 기술 방법?
 - UML 같은 Graphical Notations? → 불충분
 - 설계패턴의 적용환경, 목적, 문제점, 대안책, 장단점 등이 기술되어야 함.
- GoF의 설계패턴 Template

- Pattern Name
- Intent
- Also Known As
- Motivation
- Applicability
- Structure
- Participants

- Collaborations
- Consequences
- Implementation
- Sample Code
- Known Uses
- Related Patterns

디자인 패턴 표현 방법

- Pattern Name
 - 설계패턴을 대표하는 이름으로 일반적으로 한 두 단어로 표현
 - 설계패턴의 내용을 핵심적으로 나타낼 수 있는 것이어야 한다.
 - 설계패턴 이용자들 간의 의사전달에서 용어로 이용되기 때문에 주의하여 작성되어야 한다.
- Intent
 - 설계패턴이 무엇을 하려고 하는지를 짧은 문장으로 기술
- Also Known As
 - 해당 설계패턴에 대한 잘 알려진 다른 이름들이 있다면 기술
- Motivation
 - 특정 상황에서 설계문제를 기술하고, 패턴에 있는 클래스와 객체들이 이 문제를 어떻게 해결하는지를 예를 들어 설명한다.
- Applicability
 - 어떤 상황에서 설계패턴이 적용될 수 있는지에 대하여 설명

디자인 패턴 표현 방법

- Structure
 - 패턴에 사용되는 클래스들의 시각적 표현(graphical representation)
 - 일반적으로 UML의 class diagram 과 sequence diagram 이용
- Participants
 - 패턴에 참여하는 참여자 -클래스 or 객체- 들과 각각의 역할을 기술
- Collaborations
 - 각 참여자들이 맡은 역할을 수행하기 위하여 어떻게 상호 협력하는지 기술
- Consequences
 - 설계패턴을 적용했을 때 발생할 수 있는 결과와 trade-off에 대하여 기술

디자인 패턴 표현 방법

- Implementation
 - 설계패턴을 구현할 때 알고 있어야 될 함정(pitfalls)이나 힌트(hints), 또는 기술(techniques) 등에 대하여 기술
 - 특정 프로그래밍 언어에 의존적인지 여부 기술
- Sample Code
 - 설계패턴을 어떻게 구현하는지를 보여주는 예제 코드
- Known Uses
 - 설계패턴이 기존에 알려진 실제 시스템에서 발견된 예를 기술
- Related Patterns
 - 다른 패턴과 어떻게 관련되어 있는지를 설명한다.

디자인 패턴의 목록

- 다양한 수준의 설계패턴 존재
 - 프로그래밍 수준 $\longleftrightarrow$ 추상화된 상위계층 수준
 - 수 천 가지 이상의 패턴들이 발표되었으며, 문서와 국제회의에서 토의되고 있음
 - 일반 개발자는 새로운 설계패턴을 작성하기 보다는 이미 정리되어 있는 설계패턴을 활용하는 것이 바람직함
 - GoF에서 23개의 설계패턴을 3가지 유형으로 분류하여 목록(Catalog)화 해 놓음

디자인 패턴 목록

		Purpose		
		Creational	Structural	Behavioral
Scope	Class	Factory Method (107)	Adapter(class) (139)	Interpreter(243) Template Method(325)
	Object	Abstract Factory(87) Builder(97) Prototype(117) Singleton(127)	Adapter(object) (139) Bridge(151) Composite(163) Decorator(175) Facade(185) Flyweight(195) Proxy(207)	Chain of Responsibility(223) Command(233) Iterator(257) Mediator(273) Memento(283) Observer(293) State(305) Strategy(315) Visitor(331)